08. How the System Works: From Request to Result

Stage 1: Ingestion & Features

- **Capture request:** Read the user's goal.
- **Clean text:** Remove noise and missing values.
- **Extract features:** Convert request into 10k TF-IDF signals.

Stage 2: Intelligence & DAG

- **Classify intent:** 12 task intents (99.6% accuracy).
- **Choose agents & models:** Match sub-tasks to specialists.
- **Build the DAG:** Structure tasks by dependencies & safety.

Stage 3: Execution & Storage

- **Execute:** Run parallel tasks ($K \leq 5$) with MCP tools.
- **Stream updates:** Live SSE feedback and console logs.
- **Check & save:** ORPO verification and PrismDB storage.

How a DAG is Created: Nodes, Edges & Dependencies

1. User Request
"Research & send report"

→ **2. Split into Tasks**
Define Nodes $V = \{v_i\}$

→ **3. Add Dependencies**
Map input $\leftarrow$ output links

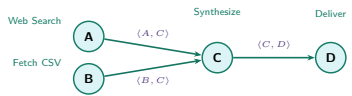
→ **4. Create DAG $\mathcal{G}(V, E)$**
Form directed edges E

→ **5. Run in Parallel**
Execute in-degree $= 0$

DAG Formulation in PrismSpace: $\mathcal{G}(V, E)$

- **Nodes / Vertices ($v \in V$):** Atomic **agent sub-tasks** (e.g., Web Search, Data Fetch, Synthesize, Deliver), each assigned an agent, model, and toolset.
- **Directed Edges ($e = \langle v_i, v_j \rangle \in E$): Data flows & dependencies**. Edge passes output artifacts/state from v_i into v_j; task v_j is blocked until v_i completes.
- **"Add Dependencies" (Step 3):** Connects sub-tasks based on prerequisite inputs (e.g., Synthesize needs Search + CSV). Tasks with in-degree $= 0$ run in parallel; others wait.
- **Acyclic ($\nexists$ cycle):** No circular deadlocks $\Rightarrow$ topological order π guarantees safe scheduling.

Concrete Workflow Example



Nodes (V): Sub-tasks A (Search), B (Data), C (Draft), D (Send)
Dependencies (E): Task C waits for A's search + B's data

Parallel start: A & B (in-deg $= 0$) run together $\rightarrow$ C $\rightarrow$ D finishes